

Don't Go Broke

Before the Agent Starts

Where This Booklet Fits:

This is booklet 3 of 5 in the Don't Go Broke series.

Disclaimer: This booklet provides general educational information and is not formal financial, legal, or professional engineering advice.

Table of Contents

- Field Note Opening
- 1. The Agent Is Not Just Typing Suggestions
- 2. The Sandbox Strategy
- 3. Decide What the Agent May Change
- 4. Decide What the Agent Must Not Touch
- 5. Define the Safe First Task
- 6. The First Safe Agent Session
- 7. Set Tool and Command Boundaries
- 8. Make the Agent Stop and Explain
- 9. Reviewing the Agent's Change Receipt
- 10. Before / During / After: The Operating Rhythm
- 11. The Bad Agent Session: A Recovery Walkthrough
- 12. The Agent Prompt Pack
- 13. The Before-the-Agent Checklist
- Field Note Closing

Before the Agent Starts

Field Note Opening

You have your blueprint. You know what you want to build. Now, you are staring at an empty chat box connected to an AI coding agent.

Your instinct is to type: *"Build the dashboard."*

Stop.

The moment before you hit "Enter" on a broad task is the most dangerous moment in your build.

If you are using an autonomous coding agent, you are no longer just receiving code suggestions. You have handed the keyboard to an incredibly fast, highly capable junior developer who never sleeps, never hesitates, and—crucially—never asks for permission unless you force it to.

Before the agent starts acting, you must define its boundaries. You must tell it what it is allowed to touch, what tools it is allowed to use, and exactly when it must stop and wait for your approval.

Speed without direction isn't progress. It's just faster chaos. This guide is about putting up the guardrails before you step on the gas.

1. The Agent Is Not Just Typing Suggestions

If you have used ChatGPT to write a Python script, or if you use GitHub Copilot to autocomplete a function, you are used to passive AI.

Passive AI types where your cursor is. It waits for you to ask a question. It gives you a block of code, and you decide where to paste it. If it gives you bad code, nothing breaks until you manually integrate it into your project.

Agentic coding tools are fundamentally different. They are autonomous.

An agent can read your entire codebase. It can open files, delete lines of code, and create new directories. It can run commands in your terminal. If it encounters an error, it doesn't wait for you to fix it; it will read the error log and try to fix the error itself.

The preparation required for a passive tool is completely different from the management required for an active agent. You are not managing a spellchecker. You are managing a worker. And if you give a fast worker a vague instruction, they will guess what you meant.

When an agent guesses, it usually guesses wrong. It will over-engineer simple solutions, install heavy libraries you don't need, and rewrite working code because it thought it was "cleaning up."

Field Note: The Danger of the Helpful Agent A builder asked an agent to "make the login button blue." The agent noticed that the CSS was written in raw CSS, not Tailwind. It decided Tailwind was a "best practice." It installed Tailwind via the terminal, rewrote all the CSS files, updated the build scripts, and broke the entire application. The button was blue, but nothing else worked.

2. The Sandbox Strategy

Because agents have the power to execute changes across multiple files simultaneously, you must never test a new, untethered agent directly on your production-ready codebase.

Before the agent starts, you need a Sandbox.

A Sandbox is an isolated environment where the agent can make mistakes, break things, and hallucinate dependencies without causing permanent damage to your core project.

How to Create a Sandbox:

- 1. Git Branches:** The simplest sandbox is a new Git branch. Never let an agent operate directly on `main` or `master`. Create a branch called `agent-experiment-dashboard`. If the agent ruins the code, you simply delete the branch and start over.
- 2. Dummy Repositories:** If you are asking the agent to establish a completely new architectural pattern (like setting up a complex database ORM), have it do the work in a completely separate, empty project folder. Once the agent proves it can build the architecture correctly, you can manually move the successful code into your real project.

If you don't use Git or version control, **do not use an AI agent**.
You must have a "Save Point" you can roll back to when the agent makes a catastrophic mistake.

3. Decide What the Agent May Change

Because an agent can touch anything in your project, your first job is to tell it exactly what it is allowed to touch today.

If you ask a human contractor to paint your kitchen, you don't want to come home and find they also demolished your bathroom because they thought the tile looked outdated. Agents will do exactly this to your code.

You must define the allowed scope in plain language before every major task.

Vague: *"Style the login button."*

Bounded: *"You may only read `src/components/Login.js` and edit `src/css/styles.css` to style the login button."*

Restrict the agent's playing field to only what is strictly necessary for the immediate task. If you are building a new UI button, the agent has no business reading your database schema or editing your routing logic.

4. Decide What the Agent Must Not Touch

Defining what the agent *can* touch is only half the battle. You must explicitly protect your core assets.

Every project has critical load-bearing code. It might be your authentication logic, your payment processing integration, or a complicated database query that finally works after three days of debugging.

If you don't understand how a piece of working code works yet, the agent should not be allowed to rewrite it.

Introduce "do not touch" boundaries early in your prompt.

Example Boundary: *"Do not modify the `auth/` directory under any circumstances. If a feature requires changing how authentication works, stop and ask me first."*

Agents love to "clean up" code. They will see a working configuration file, decide it isn't using the latest syntax, and rewrite it. Suddenly, your app won't deploy, and you have no idea why. Protect your working parts.

The `.gitignore` Defense

If you have sensitive API keys, `.env` files, or proprietary data in your project folder, do not rely purely on a text prompt to stop the agent from reading them. Ensure your `.gitignore` is properly configured. Many agents respect `.gitignore` files and will physically prevent themselves from indexing or reading ignored files.

5. Define the Safe First Task

First-time builders often treat agents like magic wands. They drop massive, multi-step requests into the prompt window.

- “Clean this up.”
- “Fix everything.”
- “Make it better.”
- “Finish the app.”
- “Add auth and deploy it.”

These are not tasks; they are wishes. And when an agent is given a wish, it will try to fulfill it all at once, opening a dozen files and tangling the logic into a knot.

You must break your blueprint down into a **Safe First Task**.

A safe first task is narrow, isolated, and highly verifiable. It has a clear definition of "done."

Unsafe: *"Build the user dashboard."* (The agent will guess the layout, guess the data structure, install charting libraries, and write fake API endpoints.)

Safe: *"Create the empty HTML and CSS layout container for the user dashboard. Do not connect it to data yet. Just build the visual shell."*

Start with a small, testable chunk of work. Verify it works. Then give the agent the next small piece.

6. The First Safe Agent Session

Let's walk through what a safe first agent request looks like from start to finish. When you are ready to execute your Safe First Task, you should follow this exact sequence:

1. **Choose a small task:** Pick one isolated feature (e.g., "Create the HTML shell for the navigation bar").
 2. **Create or confirm a branch:** Run `git checkout -b feature-navbar` so your `main` branch is protected.
 3. **Name the allowed files:** Tell the agent exactly which two or three files it is allowed to open.
 4. **Forbid dependency changes:** Explicitly state that no new packages or libraries may be installed.
 5. **Require a plan first:** Tell the agent to output its plan in plain English before writing code.
 6. **Approve the plan:** Read the plan. If the agent intends to touch a file you didn't authorize, reject the plan. If it looks correct, reply with "Approved, proceed."
 7. **Inspect the diff:** Once the agent generates the code, review the line-by-line changes in your IDE's source control tab.
 8. **Ask for a change receipt:** Have the agent summarize exactly what it accomplished.
 9. **Decide whether to keep or revert:** Run the code locally. If it works, commit it. If it broke existing features, immediately revert the branch and start over.
-

7. Set Tool and Command Boundaries

Agents are increasingly capable of running commands directly in your terminal. This is a massive productivity boost, but it is also the fastest way to break your computer.

An agent can run `npm install` to add new dependencies. It can run database migrations. It can push code to production. It can delete directories.

You must set hard boundaries around what commands the agent is allowed to run without your explicit, manual approval.

The `npm install` Warning

The most common and destructive silent action an agent takes is installing unneeded dependencies. If an agent doesn't know how to write a sorting algorithm, it will install a third-party library to do it. Over time, your simple app will become bloated with dozens of unmaintained dependencies, creating security vulnerabilities and deployment nightmares.

Example Boundary: *"You may run local development servers (like `npm run dev`) and test suites. You may NOT run any `npm install` commands, database migrations, or deployment scripts without asking me first."*

Dependency Change Review Checklist

If the agent pauses and asks for permission to install a new dependency, do not blindly approve it. Ask yourself (and the agent) these questions first:

- ☐ What specific package is being added?
- ☐ Why is it needed? (Can we write the 10 lines of code ourselves instead?)
- ☐ Is there already an existing package in our project that solves this?
- ☐ Does this package affect production deployment?
- ☐ Does it change build behavior or significantly increase the bundle size?
- ☐ Does it create an ongoing maintenance or security cost?

If you cannot answer these questions, reject the dependency and ask the agent for an alternative solution that uses standard code.

8. Make the Agent Stop and Explain

Agents will make silent architectural decisions unless you stop them.

If you tell an agent to "save user preferences," it has to decide *how* to save them. Should it use local storage? A database? A flat file? If you haven't told it how, it will choose for you. It might choose a complex, expensive database solution when all you needed was a simple text file.

You dictate the workflow rhythm. The agent stops when you say stop.

The most effective boundary you can set is an approval gate. Before the agent writes a single line of code, require it to explain its plan.

It is infinitely easier to correct a bad plan written in plain English than it is to untangle bad code spread across five different files.

9. Reviewing the Agent's Change Receipt

When the agent claims it has finished a task, the session is not over. Do not immediately move on to the next task. First, you must review the change receipt.

Ask the agent to summarize exactly what it just did, then critically evaluate its response against what you actually asked for.

Look for:

- **What files changed:** Did it touch the specific files you authorized?
- **Why they changed:** Did the logic match the approved plan?
- **What was not changed:** Did it leave your protected files alone?
- **What commands were run:** Did it sneak in a silent installation or migration?
- **What risks remain:** Does the agent mention any edge cases it ignored?
- **What you still need to verify:** What manual testing is required before you can trust this code?

If the change receipt reveals the agent touched files outside the allowed scope, you must treat the entire change as suspect.

10. Before / During / After: The Operating Rhythm

Managing an agent requires a strict operating rhythm. If you abandon this rhythm, the agent will eventually drift out of scope.

Before the Agent Runs

- **Define the task:** Choose one isolated, verifiable step.
- **Define the files:** Explicitly state the allowed file paths.
- **Define the commands:** Explicitly ban installations or destructive terminal commands.
- **Define stop rules:** Require the agent to output a plan before writing code.

During the Agent Run

- **Watch for scope drift:** Monitor the agent's output as it generates code.
- **Pause on unrelated work:** If you see the agent open a file related to authentication when it should be styling a button, press the stop button immediately.
- **Ask for explanations:** If the agent takes an unexpected path, ask "Why did you just open that file?" before allowing it to proceed.

After the Agent Runs

- **Inspect changed files:** Look at the visual diff in your version control tool.
 - **Run tests manually:** Never assume the code works just because the agent says it does. Click the buttons. Run the app.
 - **Ask what changed:** Generate the change receipt.
 - **Commit only when understood:** If you don't understand the code the agent wrote, ask it for line-by-line comments before you commit it to your repository.
-

11. The Bad Agent Session: A Recovery Walkthrough

Even with perfect boundaries, an agent will occasionally lose context, hallucinate a tool that doesn't exist, or embark on a "Silent Refactor."

The Scenario

You ask the agent to fix the padding on a button. It fixes the padding. It then notices a typo in the header and fixes the typo. It then notices the routing logic is outdated, updates the router, installs a new version of `react-router-dom` via the terminal, breaks the app, and spends the next hour trying to fix the router it just broke.

The agent claims it fixed the padding, but the app no longer boots.

The Recovery Sequence

Do not ask the agent to "fix what you just broke." The agent has lost context. Its mental map of your codebase is now corrupted by the hallucinations and unnecessary changes it just made. Asking it to fix a mess usually results in a deeper mess.

Instead, execute the recovery sequence:

1. **Stop:** Press the halt button in your IDE or terminal. Cut off the agent's execution immediately.
 2. **Inspect the Diff:** Look at your Git changes. You will see the button padding change, plus dozens of unauthorized routing changes and a modified `package.json`.
 3. **Revert:** Throw away all the agent's changes. Run `git restore .` or `git reset --hard HEAD` to return your project to the last known working Save Point.
 4. **Narrow the Scope:** Acknowledge that your initial prompt gave the agent too much leeway.
 5. **Restart with Stricter Boundaries:** Start a completely new chat thread. The old thread is poisoned with bad context. In the new thread, explicitly ban the agent from looking at or touching the router.
-

12. The Agent Prompt Pack

Do not reinvent the wheel every time you open a new chat window. Copy and paste these practical prompts directly into your agent's system instructions or initial request.

The Plan-First Prompt

"STOP AND PLAN RULE: Before you edit any files or write any code, you must output a step-by-step plan. List the exact files you intend to create or modify. List the architectural approach you intend to take. Do not write the actual code or run any commands until I reply with 'Approved'."

The File-Boundary Prompt

"ALLOWED SCOPE: You are only permitted to read and modify the following files: [List files here]. Do not open, read, or modify any other files in this project to complete this task. If you believe you need to touch another file, stop and ask me for permission."

The Command-Boundary Prompt

"COMMAND BOUNDARIES: You are authorized to run local development servers and test suites to verify your work. You are STRICTLY FORBIDDEN from running `npm install`, adding any new dependencies, running database migrations, or executing any destructive commands without asking me first."

The Stop-and-Ask Prompt

"DECISION RULE: If you encounter ambiguity, do not guess. If there are multiple architectural paths to solve a problem, do not pick one silently. Stop, list the options, and ask me which path to take."

The Change Receipt Prompt

"TASK COMPLETE RULE: When you have finished the task, output a Change Receipt. List exactly which files you modified, what commands you ran, and why. List any edge

cases or risks that I need to manually verify before committing this code."

The Recovery/Restart Prompt (Use in a fresh thread after a failure)

"We are restarting this task because the previous attempt failed due to unauthorized scope expansion. The task is [Task Name]. You are strictly forbidden from modifying [Area that broke previously]. Focus entirely on the isolated task."

13. The Before-the-Agent Checklist

Before you hit "Enter" on your first agent prompt, confirm your readiness.

- ☐ **Agent Task:** I have defined a narrow, verifiable task.
 - ☐ **Allowed Files/Folders:** I have explicitly listed which specific files the agent is allowed to edit.
 - ☐ **Off-Limits Files:** I have explicitly protected critical files the agent must never touch.
 - ☐ **Command Rules:** I have forbidden the agent from installing new dependencies or running deployments without asking.
 - ☐ **Stop Point:** I have required the agent to output a plan before writing code.
 - ☐ **Review Requirement:** I know exactly what the agent must show me before it proceeds.
 - ☐ **Definition of Done:** I know exactly how I will manually test that this specific task is complete.
 - ☐ **Sandbox Ready:** I am on a safe Git branch or working in an isolated environment.
 - ☐ **If Unsure:** I have instructed the agent to stop and ask for clarification rather than guessing.
-

Field Note Closing

The agent should not start until the boundaries are clear.

Taking five minutes to define what the agent cannot do will save you five hours of trying to figure out what it just did.

The goal of setting these boundaries isn't to slow you down. The goal is to keep you in the driver's seat. You are the builder. The agent is just the hands on the keyboard.

Set up your sandbox. Define your Safe First Task. Fill out your checklists.

When you're ready, give the agent its first instruction.

The Next Step

You are ready for the next phase. Proceed to booklet: **Before You Trust the First Draft.**